Project 1 - Regression supervised learning Regression :

Predicting Employee Salaries : Problem Statement: Predicting Employee Salaries

Project Description: Build a regression model to predict employee salaries based on factors such as job title, years of experience, education level, and performance metrics. Domain: Human Resources

In [1]:

```
import numpy as np
import pandas as pd
data = pd.read_csv('Salary Data.csv')
data
```

Out[1]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0
...	...	...	...	...	...	...
370	35.0	Female	Bachelor's	Senior Marketing Analyst	8.0	85000.0
371	43.0	Male	Master's	Director of Operations	19.0	170000.0
372	29.0	Female	Bachelor's	Junior Project Manager	2.0	40000.0
373	34.0	Male	Bachelor's	Senior Operations Coordinator	7.0	90000.0
374	44.0	Female	PhD	Senior Business Analyst	15.0	150000.0

375 rows × 6 columns

In [3]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 375 entries, 0 to 374
Data columns (total 6 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Age                  373 non-null    float64
 1   Gender               373 non-null    object 
 2   Education Level      373 non-null    object 
 3   Job Title            373 non-null    object 
 4   Years of Experience  373 non-null    float64
 5   Salary               373 non-null    float64
dtypes: float64(3), object(3)
memory usage: 17.7+ KB
```

In [4]:

```
data.isnull().sum()
```

Out[4]:

```
Age                    2
Gender                 2
Education Level        2
Job Title              2
Years of Experience    2
Salary                 2
dtype: int64
```

In [5]:

```
data.dropna(inplace = True)
```

In [6]:

```
data.isna().sum()
```

Out[6]:

```
Age                    0
Gender                 0
Education Level        0
Job Title              0
Years of Experience    0
Salary                 0
dtype: int64
```

In [7]:

```
data.describe()
```

Out[7]:

	Age	Years of Experience	Salary
count	373.000000	373.000000	373.000000
mean	37.431635	10.030831	100577.345845
std	7.069073	6.557007	48240.013482
min	23.000000	0.000000	350.000000
25%	31.000000	4.000000	55000.000000
50%	36.000000	9.000000	95000.000000
75%	44.000000	15.000000	140000.000000
max	53.000000	25.000000	250000.000000

In [8]:

```
data.head(10)
```

Out[8]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0
5	29.0	Male	Bachelor's	Marketing Analyst	2.0	55000.0
6	42.0	Female	Master's	Product Manager	12.0	120000.0
7	31.0	Male	Bachelor's	Sales Manager	4.0	80000.0
8	26.0	Female	Bachelor's	Marketing Coordinator	1.0	45000.0
9	38.0	Male	PhD	Senior Scientist	10.0	110000.0

In [9]:

```
data.tail(10)
```

Out[9]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
365	43.0	Male	Master's	Director of Marketing	18.0	170000.0
366	31.0	Female	Bachelor's	Junior Financial Analyst	3.0	50000.0
367	41.0	Male	Bachelor's	Senior Product Manager	14.0	150000.0
368	44.0	Female	PhD	Senior Data Engineer	16.0	160000.0
369	33.0	Male	Bachelor's	Junior Business Analyst	4.0	60000.0
370	35.0	Female	Bachelor's	Senior Marketing Analyst	8.0	85000.0
371	43.0	Male	Master's	Director of Operations	19.0	170000.0
372	29.0	Female	Bachelor's	Junior Project Manager	2.0	40000.0
373	34.0	Male	Bachelor's	Senior Operations Coordinator	7.0	90000.0
374	44.0	Female	PhD	Senior Business Analyst	15.0	150000.0

In [10]:

```
data.columns
```

Out[10]:

```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experien
ce',
       'Salary'],
      dtype='object')
```

In [12]:

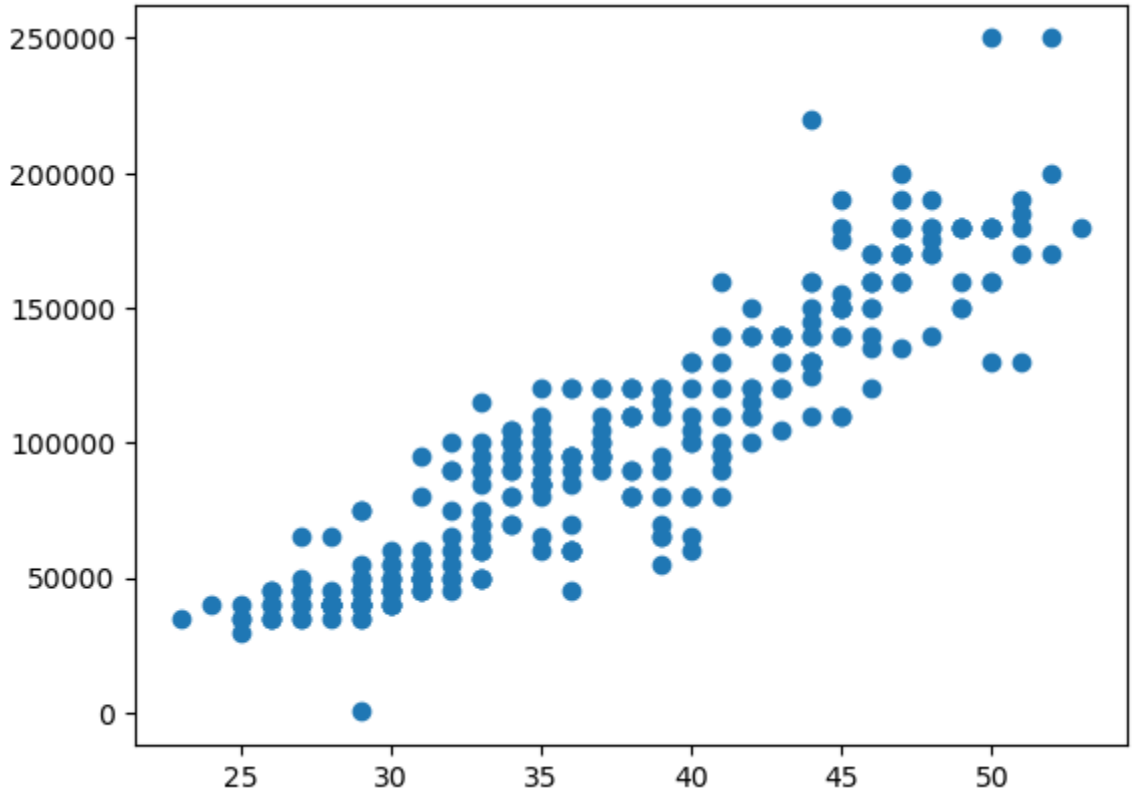
```
####Plotting Scatter plot to check relationship between independent variable and dependen
import matplotlib.pyplot as plt
```

In [13]:

```
plt.scatter(data['Age'][:300], data["Salary"][:300])
```

Out[13]:

```
<matplotlib.collections.PathCollection at 0x27c42f3e040>
```

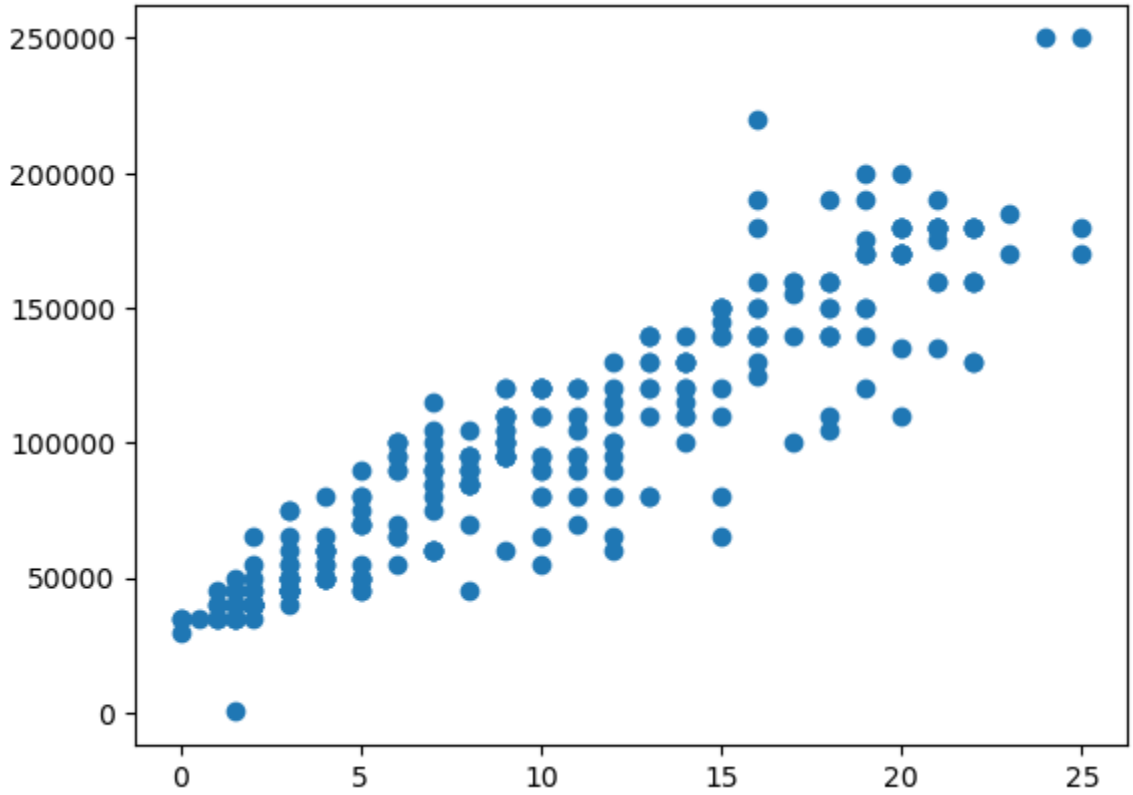


In [14]:

```
plt.scatter(data['Years of Experience'][:300], data["Salary"][:300])
```

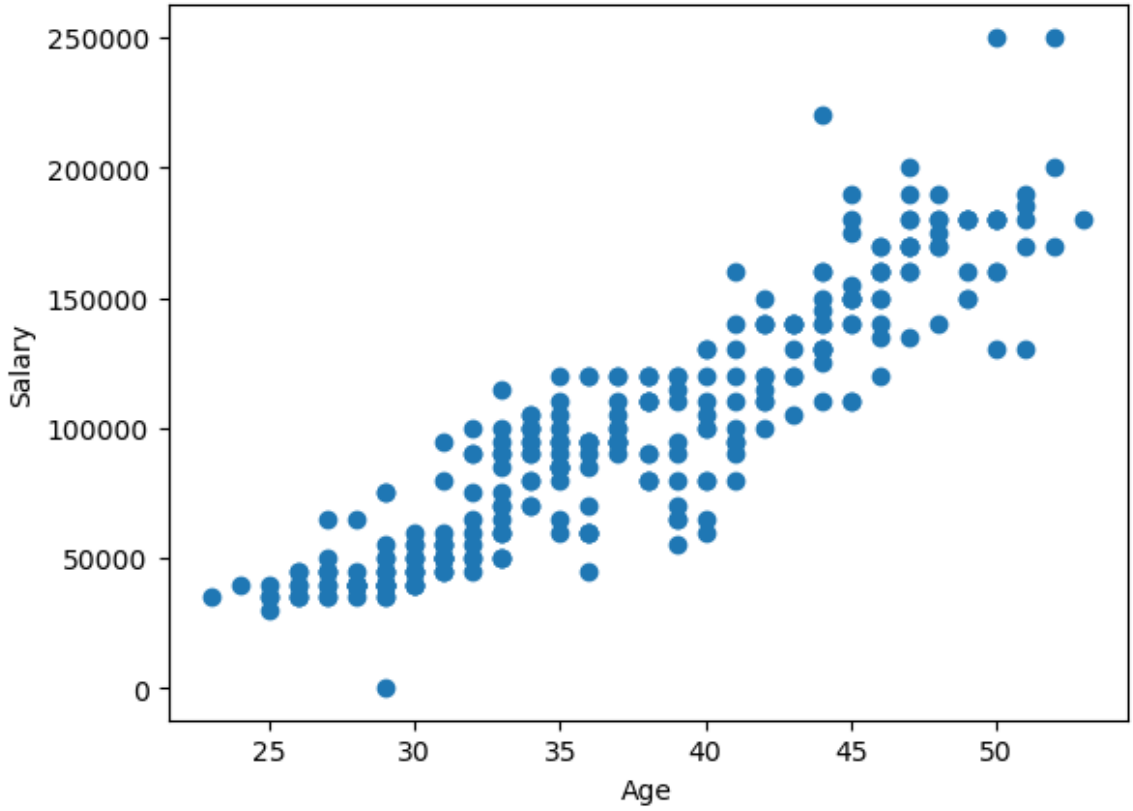
Out[14]:

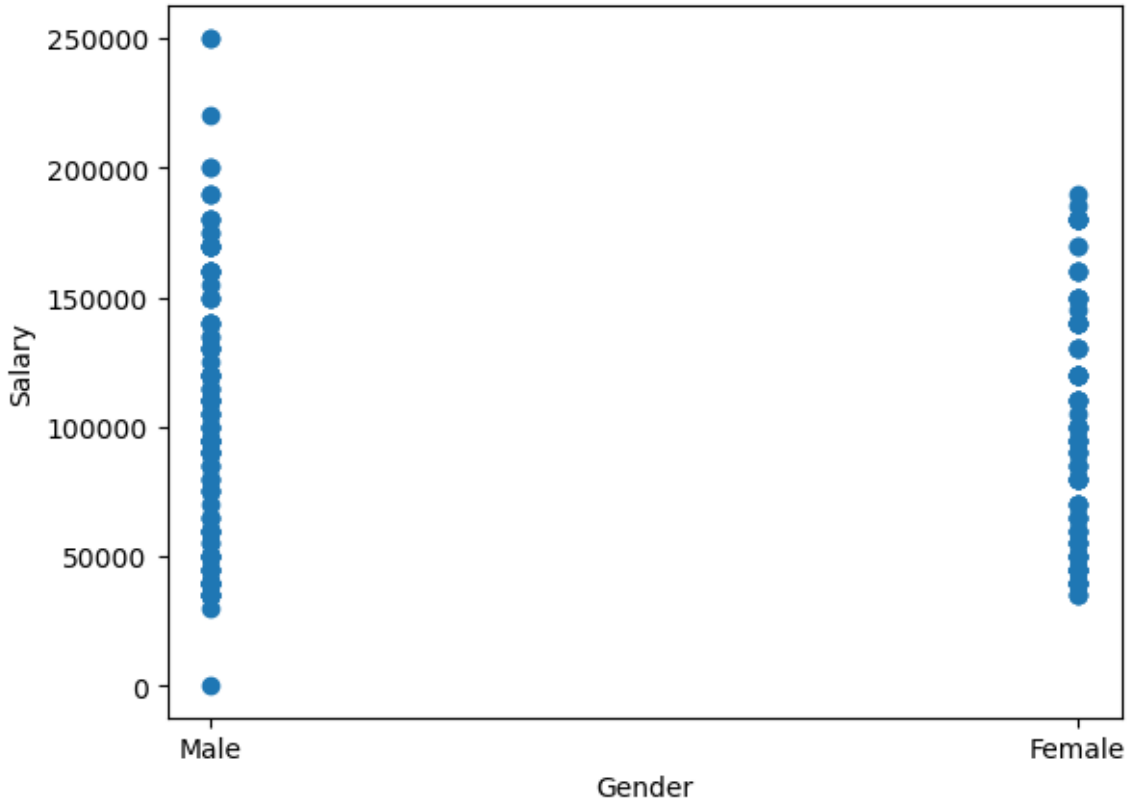
```
<matplotlib.collections.PathCollection at 0x27c438345b0>
```

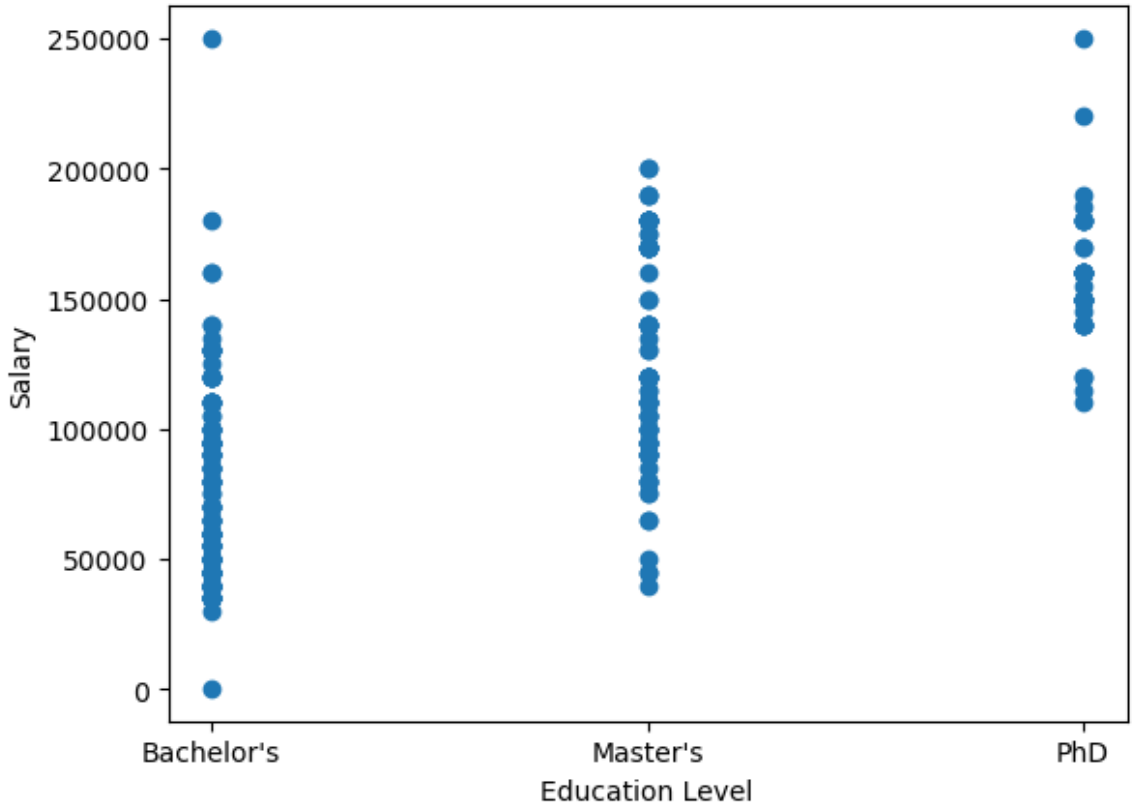


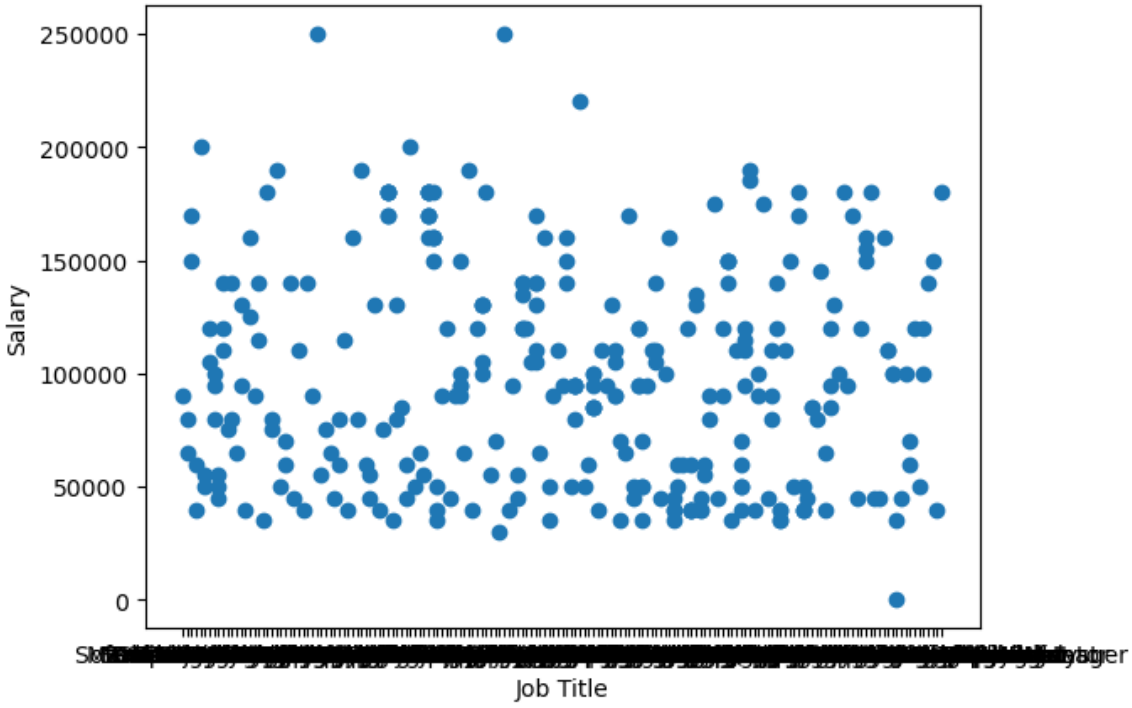
In [15]:

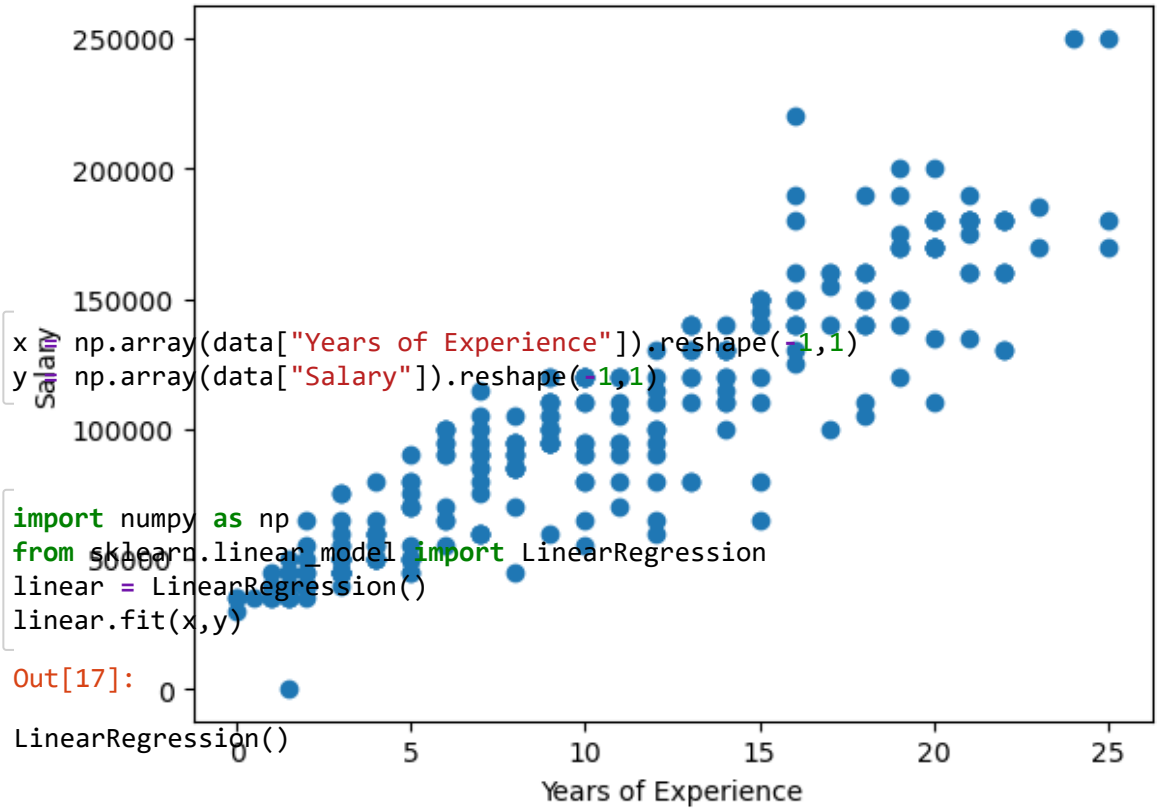
```
for i in data.columns[:-1]:
    plt.xlabel(i)
    plt.ylabel("Salary")
    plt.scatter(data[i][:300],data["Salary"][:300])
    plt.show()
```






250000
200000
150000
100000
50000
0
Salary
Bachelor's
Master's
PhD
Education Level


250000
200000
150000
100000
50000
0
Salary
Job Title




```
x = np.array(data["Years of Experience"]).reshape(-1,1)
y = np.array(data["Salary"]).reshape(-1,1)
```

```
import numpy as np
from sklearn.linear_model import LinearRegression
linear = LinearRegression()
linear.fit(x,y)
```

Out[17]:

```
LinearRegression()
```

In [18]:

```
linear.predict([[48]])
```

Out[18]:

```
array([[360457.72161228]])
```

multiple regression

In [19]:

```
len(data.columns)
```

Out[19]:

```
6
```

In [20]:

```
data.columns
```

Out[20]:

```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experien
ce',
       'Salary'],
      dtype='object')
```

In [31]:

```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Gender']= label_encoder.fit_transform(data['Gender'])

data['Gender'].unique()
```

Out[31]:

```
array([1, 0])
```

In [44]:

```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Education Level']= label_encoder.fit_transform(data['Education Level'])

data['Education Level'].unique()
```

Out[44]:

```
array([0, 1, 2])
```

In [51]:

```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Job Title']= label_encoder.fit_transform(data['Job Title'])

data['Job Title'].unique()
```

Out[51]:

```
array([159,  17, 130, 101,  22,  81,  93, 104,  82, 150, 158,  40,  36,
        96,  13,  89,  83, 116,  18, 102,   3, 172,  44,  98,  37, 157,
       160,  57, 112,  92,   6,   1,  19,  84, 167,  39,  95,  16, 103,
       169,  88,  85,   2, 162,  10,   0,  29,  41,  12,   5,  34, 171,
        38, 170, 156,  30, 115,  47,  21,  43,  14,   4, 118, 173,  99,
       166,  11, 153,  42,   9, 165, 106,   8,  56,  35,  46, 144,  91,
       164, 134, 168, 100,  76,  97,  87,  94, 122,  80, 143,   7,  20,
        45, 131,  15, 120, 161, 163, 110,  62,  86,  90,  61, 141,  66,
       121, 105, 152,  79, 154, 146,  73,  64,  54, 142,  49, 148,  65,
        70, 109,  32,  71, 111, 139,  53, 129,  59, 138,  28,  77, 149,
        31,  52, 132, 125,  50, 108, 147,  60,  25,  63,  55, 136, 124,
       155,  69, 135, 128, 145,  33, 107,  23,  74, 126, 113,  27,  48,
        78, 133, 127, 117,  51,  75, 140,  68, 151,  72, 119, 123, 114,
        67,  26, 137,  58,  24])
```

In [52]:

```
data
```

Out[52]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	1	0	159	5.0	90000.0
1	28.0	0	1	17	3.0	65000.0
2	45.0	1	2	130	15.0	150000.0
3	36.0	0	0	101	7.0	60000.0
4	52.0	1	1	22	20.0	200000.0
...	...	...	...	...	...	...
370	35.0	0	0	131	8.0	85000.0
371	43.0	1	1	30	19.0	170000.0
372	29.0	0	0	70	2.0	40000.0
373	34.0	1	0	137	7.0	90000.0
374	44.0	0	2	110	15.0	150000.0

373 rows × 6 columns

In [53]:

```
x = data.drop("Salary",axis=1)

y = data["Salary"]
```

In [54]:

```
x.head()
```

Out[54]:

	Age	Gender	Education Level	Job Title	Years of Experience
0	32.0	1	0	159	5.0
1	28.0	0	1	17	3.0
2	45.0	1	2	130	15.0
3	36.0	0	0	101	7.0
4	52.0	1	1	22	20.0

In [55]:

```
y.head()
```

Out[55]:

```
0     90000.0
1     65000.0
2    150000.0
3     60000.0
4    200000.0
Name: Salary, dtype: float64
```

In [56]:

```
model = LinearRegression()
```

In [57]:

```
model.fit(x,y)
```

Out[57]:

```
LinearRegression()
```

In [58]:

```
model.coef_
```

Out[58]:

```
array([ 2652.29501743,  8322.28165554, 13290.95033734,    27.3298712 ,
        3156.92278531])
```

In [59]:

```
model.intercept_
```

Out[59]:

```
-44282.390111837245
```

In [62]:

```
x.columns
```

Out[62]:

```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experien
ce'], dtype='object')
```

In [63]:

```
model.predict([[3, 5, 2, 5, 16]])
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarnin
g: X does not have valid feature names, but LinearRegression was fitted wi
th feature names
  warnings.warn(
```

Out[63]:

```
array([82515.21781381])
```

In [ ]:

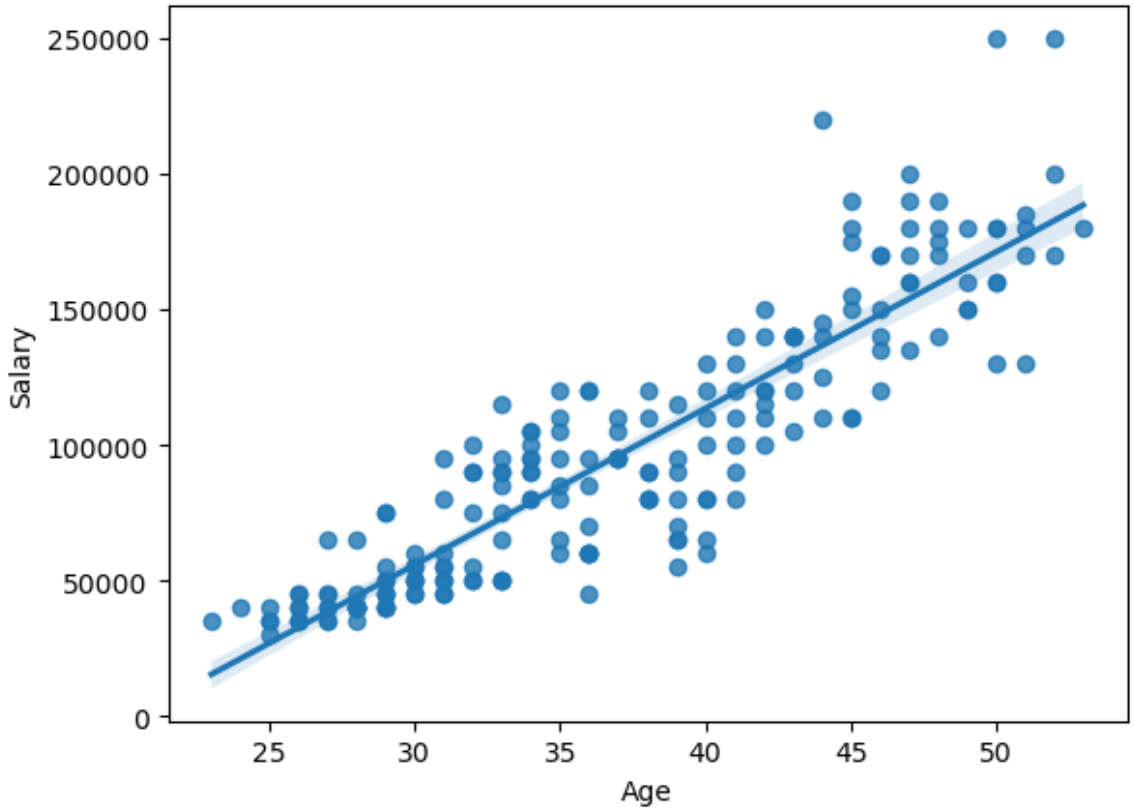
```
Salary and Age
```

In [65]:

```
import seaborn as sns
sns.regplot(x="Age",
            y = "Salary",
            data = data[:200])
```

Out[65]:

```
<AxesSubplot:xlabel='Age', ylabel='Salary'>
```



In [67]:

```
# split the data into testing and training

from sklearn.model_selection import train_test_split
```

In [68]:

```
xtrain, xtest, ytrain , ytest = train_test_split(data.drop("Salary",axis=1),data["Salary"
```

In [69]:

```
xtrain.shape
```

Out[69]:

```
(279, 5)
```

In [70]:

```
ytrain.shape
```

Out[70]:

```
(279,)
```

In [71]:

```
xtest.shape
```

Out[71]:

```
(94, 5)
```

In [72]:

```
ytrain.shape
```

Out[72]:

```
(279,)
```

In [73]:

```
xtest.shape
```

Out[73]:

```
(94, 5)
```

In [74]:

```
ytest.shape
```

Out[74]:

```
(94,)
```

In [ ]:

```
model = LinearRegression()
```

In [75]:

```
model.fit(xtrain,ytrain)
```

Out[75]:

```
LinearRegression()
```

In [76]:

```
y_pred = model.predict(xtest)
```

In [78]:

```
ytest.head(10)
```

Out[78]:

```
133    160000.0
196     90000.0
244     70000.0
210    110000.0
118     45000.0
93     170000.0
279    180000.0
173    140000.0
286     60000.0
295     60000.0
Name: Salary, dtype: float64
```

In [79]:

```
xtest.head(10)
```

Out[79]:

	Age	Gender	Education Level	Job Title	Years of Experience
133	47.0	1	2	146	22.0
196	41.0	0	0	132	11.0
244	34.0	0	0	62	6.0
210	38.0	1	1	127	9.0
118	29.0	0	0	61	3.0
93	52.0	0	1	134	25.0
279	49.0	0	1	30	21.0
173	43.0	1	0	144	16.0
286	33.0	1	0	49	4.0
295	33.0	1	0	59	4.0

In [80]:

```
for i in y_pred[:10]:
    print(i)
```

```
187888.09764677595
104165.0623589586
68223.95775895413
110358.44831589288
44716.708794075326
190337.08073629846
168849.51299774245
131861.745183982
66821.59174659693
66933.25283464388
```

In [81]:

```
model.score(xtrain, ytrain )
```

Out[81]:

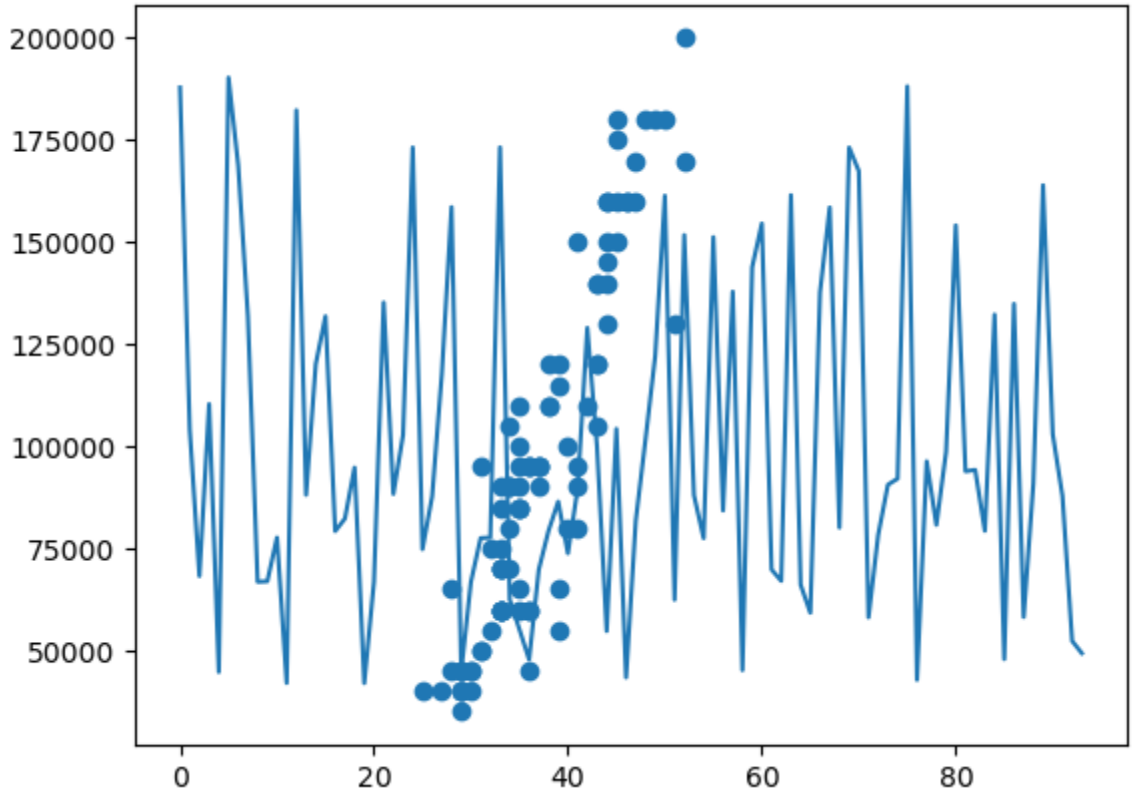
```
0.9042698954451687
```

In [82]:

```
plt.scatter(xtest["Age"][:100],ytest[:100])
plt.plot(y_pred[:100])
```

Out[82]:

```
[<matplotlib.lines.Line2D at 0x27c46340f10>]
```



In [84]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, mean_squared_log_err
```

In [85]:

```
mean_absolute_error(ytest, y_pred)
```

Out[85]:

```
10962.464182995238
```

In [86]:

```
mean_squared_error(ytest, y_pred)
```

Out[86]:

```
209417535.16259235
```

In [87]:

```
model.score(xtrain,ytrain)
```

Out[87]:

```
0.9042698954451687
```

In [ ]:

```

```

Project 2 - Classification supervised learning :

Adult Census Income Prediction : Project Title Adult Census Income Prediction Domain Finance

Problem Statement: The Goal is to predict whether a person has an income of more than 50K a year or not. This is basically a binary classification problem where a person is classified into two groups greater than 50K group or less than or equals to 50K group.

In [10]:

```
import numpy as np
import pandas as pd
data1 = pd.read_csv('adult.csv')
data1
```

2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female
...	...	...	...	...	...	...	...	...	...	...
32556	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support	Wife	White	Female
32557	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male
32558	58	Private	151910	HS-grad	9	Widowed	Adm-clerical	Unmarried	White	Female

In [11]:

```
data1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education-num   32561 non-null  int64
 5   marital-status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital-gain    32561 non-null  int64
 11  capital-loss    32561 non-null  int64
 12  hours-per-week  32561 non-null  int64
 13  country         32561 non-null  object
 14  salary          32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

In [12]:

```
data1.head()
```

Out[12]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black

In [13]:

```
data1.dtypes
```

Out[13]:

```
age                int64
workclass         object
fnlwgt             int64
education         object
education-num      int64
marital-status    object
occupation        object
relationship      object
race              object
sex               object
capital-gain       int64
capital-loss       int64
hours-per-week     int64
country           object
salary            object
dtype: object
```

In [14]:

```
data1.describe()
```

Out[14]:

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week
count	32561.000000	3.256100e+04	32561.000000	32561.000000	32561.000000	32561.000000
mean	38.581647	1.897784e+05	10.080679	1077.648844	87.303830	40.437456
std	13.640433	1.055500e+05	2.572720	7385.292085	402.960219	12.347429
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.178270e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.783560e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.370510e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	99.000000

In [15]:

```
data1.isnull().sum()
```

Out[15]:

```
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
country           0
salary            0
dtype: int64
```

In [16]:

```
data1.dropna(inplace = True)
```

In [17]:

```
data1.isna().sum()
```

Out[17]:

```
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
country           0
salary            0
dtype: int64
```

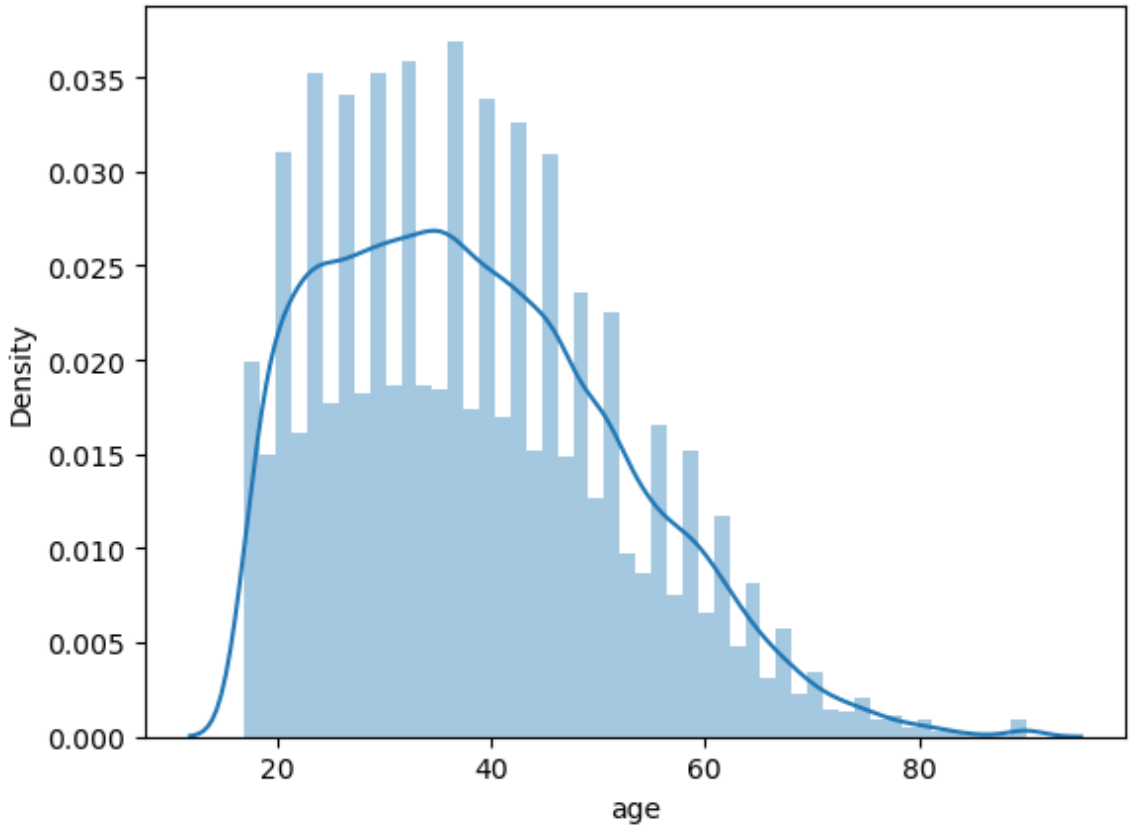
In [18]:

```
import seaborn as sns
sns.distplot(data1.age)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\distributions.py:2619:
FutureWarning: `distplot` is a deprecated function and will be removed in
a future version. Please adapt your code to use either `displot` (a figure
-level function with similar flexibility) or `histplot` (an axes-level fun
ction for histograms).
  warnings.warn(msg, FutureWarning)
```

Out[18]:

```
<AxesSubplot:xlabel='age', ylabel='Density'>
```



In [19]:

```
data1.workclass.value_counts()
```

Out[19]:

```
 Private             22696
 Self-emp-not-inc     2541
 Local-gov            2093
 ?                    1836
 State-gov            1298
 Self-emp-inc         1116
 Federal-gov           960
 Without-pay            14
 Never-worked            7
Name: workclass, dtype: int64
```

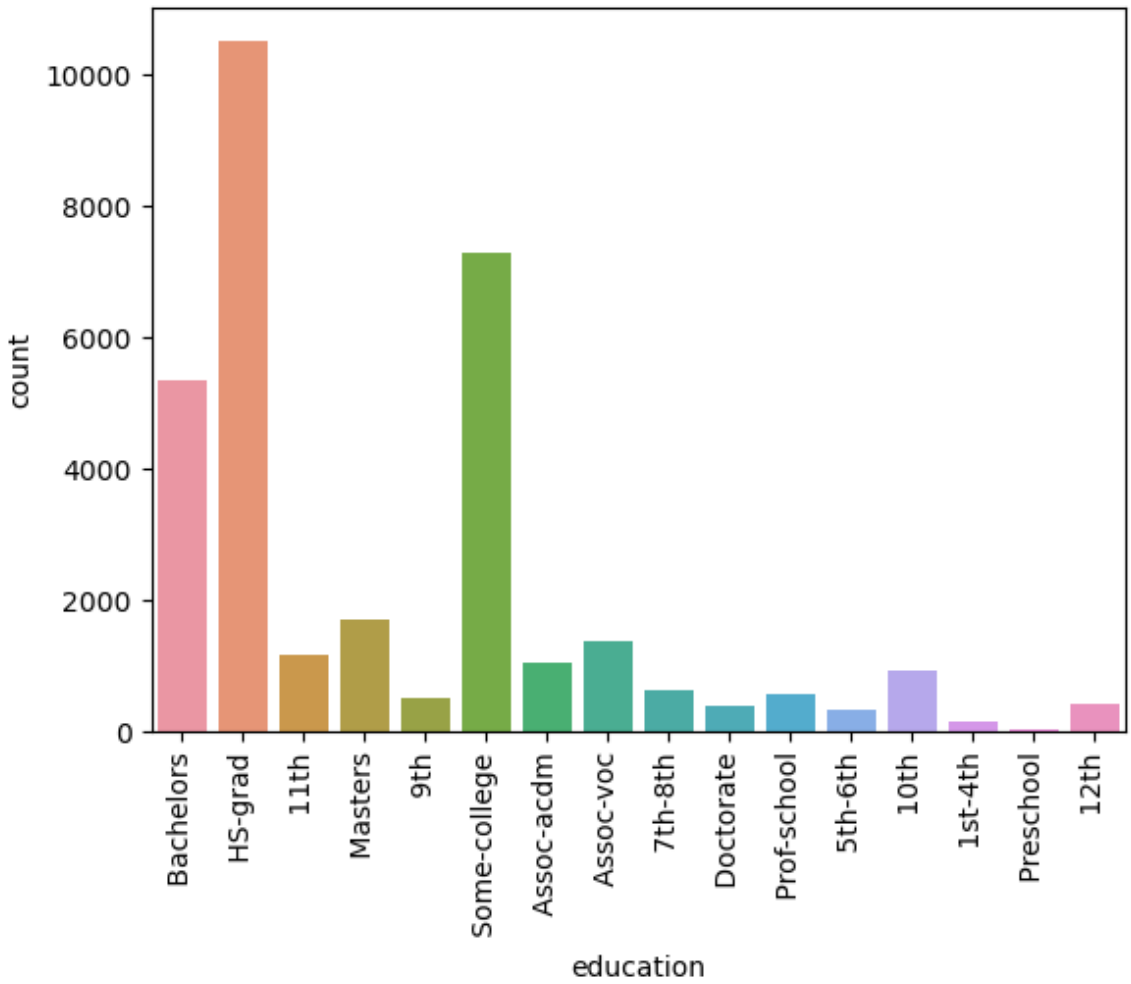
In [20]:

```
import matplotlib.pyplot as plt
sns.countplot(data1.education)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[20]:

```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15]),
 [Text(0, 0, ' Bachelors'),
  Text(1, 0, ' HS-grad'),
  Text(2, 0, ' 11th'),
  Text(3, 0, ' Masters'),
  Text(4, 0, ' 9th'),
  Text(5, 0, ' Some-college'),
  Text(6, 0, ' Assoc-acdm'),
  Text(7, 0, ' Assoc-voc'),
  Text(8, 0, ' 7th-8th'),
  Text(9, 0, ' Doctorate'),
  Text(10, 0, ' Prof-school'),
  Text(11, 0, ' 5th-6th'),
  Text(12, 0, ' 10th'),
  Text(13, 0, ' 1st-4th'),
  Text(14, 0, ' Preschool'),
  Text(15, 0, ' 12th')])
```


10000
8000
6000
4000
2000
0
count
Bachelors
HS-grad
11th
Masters
9th
Some-college
Assoc-acdm
Assoc-voc
7th-8th
Doctorate
Prof-school
5th-6th
10th
1st-4th
Preschool
12th
education

In [21]:

```
sns.countplot(data1.occupation)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[21]:

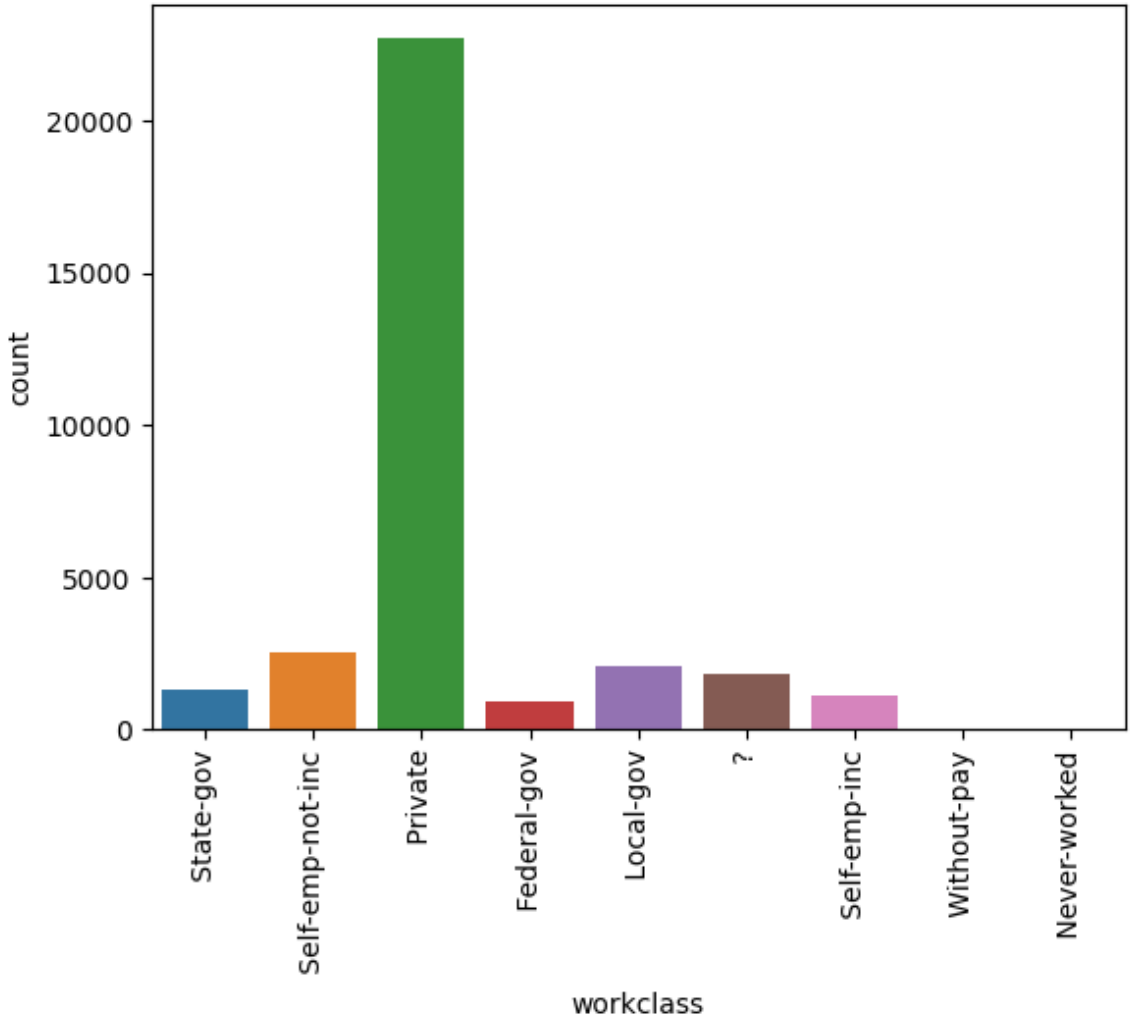
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14]),
 [Text(0, 0, ' Adm-clerical'),
  Text(1, 0, ' Exec-managerial'),
  Text(2, 0, ' Handlers-cleaners'),
  Text(3, 0, ' Prof-specialty'),
  Text(4, 0, ' Other-service'),
  Text(5, 0, ' Sales'),
  Text(6, 0, ' Craft-repair'),
  Text(7, 0, ' Transport-moving'),
  Text(8, 0, ' Farming-fishing'),
  Text(9, 0, ' Machine-op-inspct'),
  Text(10, 0, ' Tech-support'),
  Text(11, 0, ' ?'),
  Text(12, 0, ' Protective-serv'),
  Text(13, 0, ' Armed-Forces'),
  Text(14, 0, ' Priv-house-serv')])
```

```
sns.countplot(data1.workclass)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[22]:

```
(array([0, 1, 2, 3, 4, 5, 6, 7, 8]),
 [Text(0, 0, ' State-gov'),
  Text(1, 0, ' Self-emp-not-inc'),
  Text(2, 0, ' Private'),
  Text(3, 0, ' Federal-gov'),
  Text(4, 0, ' Local-gov'),
  Text(5, 0, ' ?'),
  Text(6, 0, ' Self-emp-inc'),
  Text(7, 0, ' Without-pay'),
  Text(8, 0, ' Never-worked')])
```



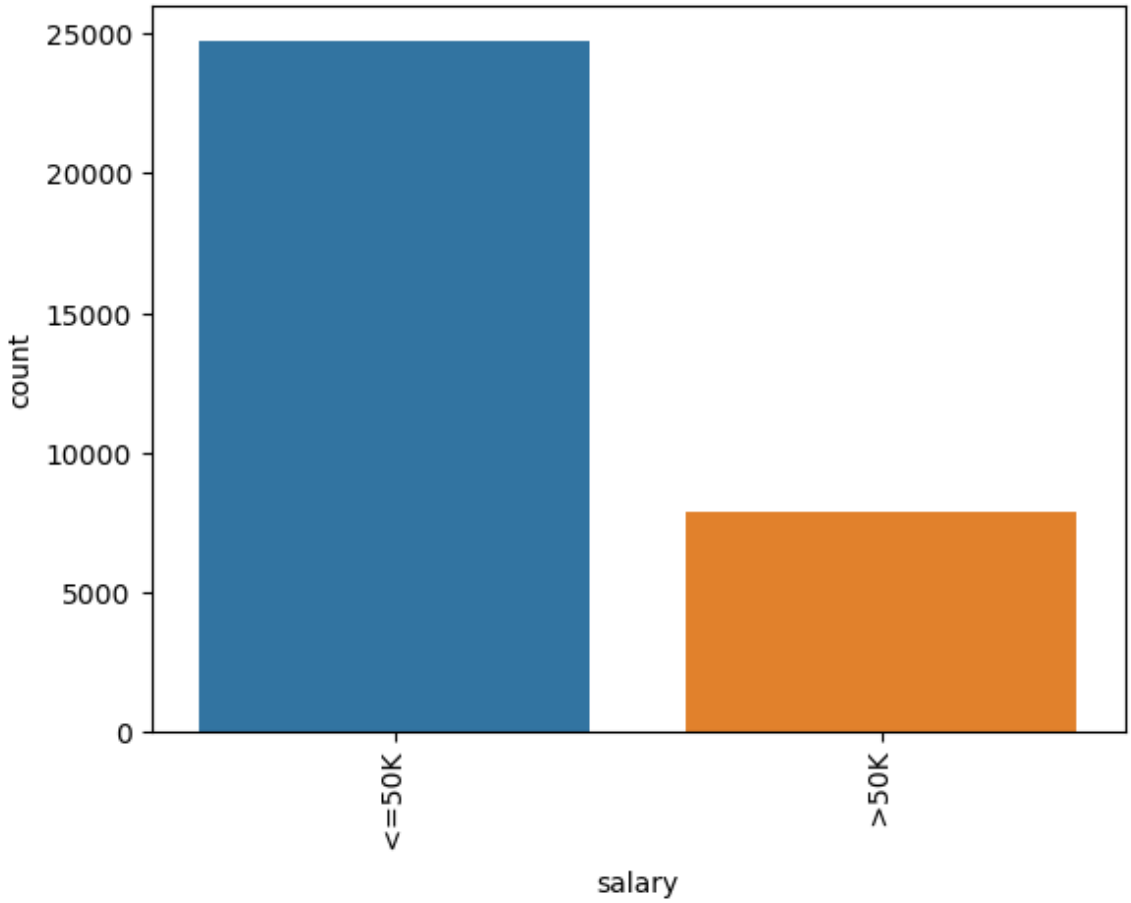
In [23]:

```
sns.countplot(data1.salary)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: Futu
reWarning: Pass the following variable as a keyword arg: x. From version
0.12, the only valid positional argument will be `data`, and passing other
arguments without an explicit keyword will result in an error or misinterp
retation.
  warnings.warn(
```

Out[23]:

```
(array([0, 1]), [Text(0, 0, ' <=50K'), Text(1, 0, ' >50K')])
```

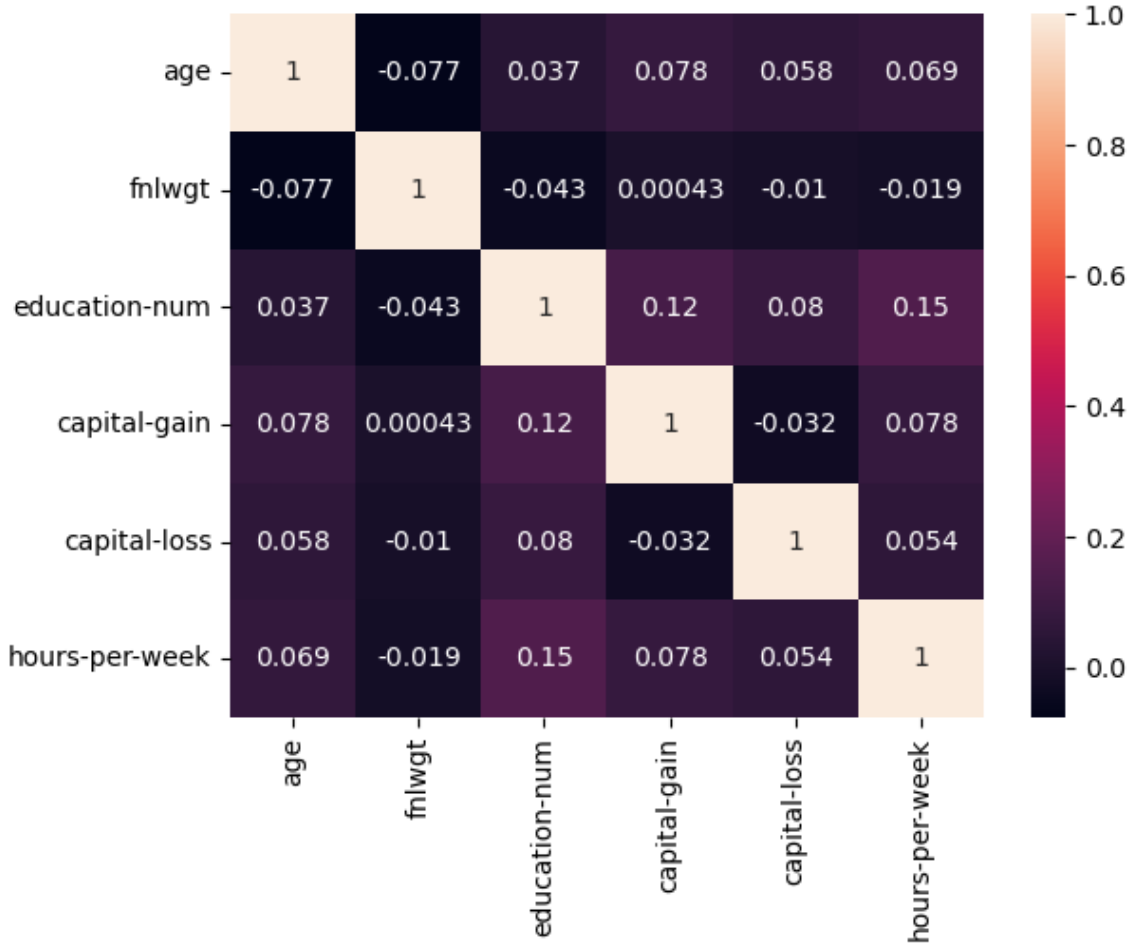


In [24]:

```
sns.heatmap(data1.corr(),annot = True)
```

Out[24]:

```
<AxesSubplot:>
```



In [ ]:

```

```

In [25]:

```
#ENCODING DATA
```

In [26]:

```
#### Import label encoder
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data1.columns
```

Out[26]:

```
Index(['age', 'workclass', 'fnlwgt', 'education', 'education-num',
       'marital-status', 'occupation', 'relationship', 'race', 'sex',
       'capital-gain', 'capital-loss', 'hours-per-week', 'country', 'salar
y'],
      dtype='object')
```

In [27]:

```
data1['workclass'] = le.fit_transform(data1['workclass'])
data1['education'] = le.fit_transform(data1['education'])
data1['workclass'] = le.fit_transform(data1['workclass'])

data1['race'] = le.fit_transform(data1['race'])
data1['sex'] = le.fit_transform(data1['sex'])
data1['relationship'] = le.fit_transform(data1['relationship'])
data1['occupation'] = le.fit_transform(data1['occupation'])
data1['country'] = le.fit_transform(data1['country'])
data1['marital-status'] = le.fit_transform(data1['marital-status'])
```

In [28]:

```
data1
```

Out[28]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race
0	39	7	77516	9	13	4	1	1	4
1	50	6	83311	9	13	2	4	0	4
2	38	4	215646	11	9	0	6	1	4
3	53	4	234721	1	7	2	6	0	2
4	28	4	338409	9	13	2	10	5	2
...	...	...	...	...	...	...	...	...	...
32556	27	4	257302	7	12	2	13	5	4
32557	40	4	154374	11	9	2	7	0	4
32558	58	4	151910	11	9	6	1	4	4
32559	22	4	201490	11	9	4	1	3	4
32560	52	5	287927	11	9	2	4	5	4

32561 rows × 15 columns

In [29]:

```
import seaborn as sns
sns.pairplot(data1)
```

Out[29]:

```
<seaborn.axisgrid.PairGrid at 0x2dc68f66340>
```

In [41]:

```
from sklearn.linear_model import LogisticRegression
```

In [42]:

```
log = LogisticRegression()
```

In [43]:

```
x = data1.drop('salary',axis = 1)
y = data1['salary']
```

In [44]:

```
x
```

Out[44]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race
0	39	7	77516	9	13	4	1	1	4
1	50	6	83311	9	13	2	4	0	4
2	38	4	215646	11	9	0	6	1	4
3	53	4	234721	1	7	2	6	0	2
4	28	4	338409	9	13	2	10	5	2
...	...	...	...	...	...	...	...	...	...
32556	27	4	257302	7	12	2	13	5	4
32557	40	4	154374	11	9	2	7	0	4
32558	58	4	151910	11	9	6	1	4	4
32559	22	4	201490	11	9	4	1	3	4
32560	52	5	287927	11	9	2	4	5	4

32561 rows × 14 columns

In [45]:

```
y
```

Out[45]:

```
0         <=50K
1         <=50K
2         <=50K
3         <=50K
4         <=50K
          ...
32556     <=50K
32557      >50K
32558     <=50K
32559     <=50K
32560      >50K
Name: salary, Length: 32561, dtype: object
```

In [46]:

```
log.fit(x,y)
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\linear_model\_logistic.
py:814: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown i
n:
    https://scikit-learn.org/stable/modules/preprocessing.html (https://sc
ikit-learn.org/stable/modules/preprocessing.html)
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-reg
ression (https://scikit-learn.org/stable/modules/linear_model.html#logisti
c-regression)
  n_iter_i = _check_optimize_result(
```

Out[46]:

```
LogisticRegression()
```

In [47]:

```
log.intercept_
```

Out[47]:

```
array([-0.00161142])
```

In [48]:

```
log.coef_
```

Out[48]:

```
array([[ 1.59636114e-02, -4.51830642e-03, -2.22771757e-06,
        -8.90242270e-03,  3.75924197e-03, -1.19130884e-02,
        -2.48600399e-03, -1.26135269e-02, -4.31213443e-03,
         1.54599766e-03,  3.21794230e-04,  7.72277773e-04,
         9.81564467e-03, -5.05935091e-02]])
```

In [49]:

```
log.score(x,y)
```

Out[49]:

```
0.7887042781241362
```

In [50]:

```
x.columns
```

Out[50]:

```
Index(['age', 'workclass', 'fnlwgt', 'education', 'education-num',
       'marital-status', 'occupation', 'relationship', 'race', 'sex',
       'capital-gain', 'capital-loss', 'hours-per-week', 'country'],
      dtype='object')
```

In [51]:

```
log.predict([[1,1,5,3,1,1,21561,23333,500000,12,0,10,5,7]])
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarnin
g: X does not have valid feature names, but LogisticRegression was fitted
with feature names
  warnings.warn(
```

Out[51]:

```
array([' <=50K'], dtype=object)
```

In [52]:

```
### navi bayes
from sklearn.naive_bayes import GaussianNB
```

In [53]:

```
nb = GaussianNB()
```

In [ ]:

```

import numpy as np
x = np.reshape(x,(-1,1))
y = np.reshape(y,(-1,1))
```

In [55]:

```
nb.fit(x,y)
```

Out[55]:

```
GaussianNB()
```

In [56]:

```
nb.var_
```

Out[56]:

```
array([[2.07695385e+02, 1.32957768e+01, 1.13380154e+10, 2.82012868e+01,
        1.70750268e+01, 1.37767577e+01, 2.92498808e+01, 1.35365939e+01,
        1.19132894e+01, 1.13779174e+01, 9.27610940e+05, 9.65763820e+04,
        1.62891946e+02, 7.29828779e+01],
       [1.21776288e+02, 1.31246361e+01, 1.05134747e+10, 1.91626934e+01,
        1.68285687e+01, 1.18757307e+01, 2.78853403e+01, 1.36319219e+01,
        1.16803096e+01, 1.12682099e+01, 2.12268879e+08, 3.54571367e+05,
        1.32410516e+02, 7.02915509e+01]])
```

In [57]:

```
nb.score(x,y)
```

Out[57]:

```
0.7950308651454193
```

In [58]:

```
x
```

Out[58]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race
0	39	7	77516	9	13	4	1	1	4
1	50	6	83311	9	13	2	4	0	4
2	38	4	215646	11	9	0	6	1	4
3	53	4	234721	1	7	2	6	0	2
4	28	4	338409	9	13	2	10	5	2
...	...	...	...	...	...	...	...	...	...
32556	27	4	257302	7	12	2	13	5	4
32557	40	4	154374	11	9	2	7	0	4
32558	58	4	151910	11	9	6	1	4	4
32559	22	4	201490	11	9	4	1	3	4
32560	52	5	287927	11	9	2	4	5	4

32561 rows × 14 columns

In [59]:

```
## svm
from sklearn.svm import SVC
```

In [60]:

```
svr = SVC()
```

In [61]:

```
svr.fit(x,y)
```

Out[61]:

```
SVC()
```

In [62]:

```
svr.predict([[1,1,5,3,1,1,21561,23333,500000,12,0,2,3,8]])
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarnin
g: X does not have valid feature names, but SVC was fitted with feature na
mes
  warnings.warn(
```

Out[62]:

```
array([' <=50K'], dtype=object)
```

In [63]:

```
svr.score(x,y)
```

Out[63]:

```
0.7967814256318909
```

In [64]:

```
from sklearn.model_selection import train_test_split
```

In [65]:

```
xtrain,xtest, ytrain,ytest = train_test_split(x,y,train_size=.75)
```

In [66]:

```
x
```

Out[66]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race
0	39	7	77516	9	13	4	1	1	4
1	50	6	83311	9	13	2	4	0	4
2	38	4	215646	11	9	0	6	1	4
3	53	4	234721	1	7	2	6	0	2
4	28	4	338409	9	13	2	10	5	2
...	...	...	...	...	...	...	...	...	...
32556	27	4	257302	7	12	2	13	5	4
32557	40	4	154374	11	9	2	7	0	4
32558	58	4	151910	11	9	6	1	4	4
32559	22	4	201490	11	9	4	1	3	4
32560	52	5	287927	11	9	2	4	5	4

32561 rows × 14 columns

In [67]:

```
y
```

Out[67]:

```
0         <=50K
1         <=50K
2         <=50K
3         <=50K
4         <=50K
          ...
32556     <=50K
32557      >50K
32558     <=50K
32559     <=50K
32560      >50K
Name: salary, Length: 32561, dtype: object
```

In [68]:

```
ytrain.shape
```

Out[68]:

```
(24420,)
```

In [69]:

```
svc = SVC()

svc.fit(xtrain,ytrain)

y_pred = svc.predict(xtest)
```

In [70]:

```
from sklearn.metrics import confusion_matrix
```

In [71]:

```
confusion_matrix(ytest, y_pred)
```

Out[71]:

```
array([[6181,    2],
       [1684,  274]], dtype=int64)
```

In [72]:

```
svc.score(xtrain, ytrain)
```

Out[72]:

```
0.795004095004095
```

In [73]:

```
svc.score(xtest, ytest)
```

Out[73]:

```
0.7929001351185359
```

In [74]:

```
from sklearn.tree import DecisionTreeClassifier
```

In [75]:

```
dtc = DecisionTreeClassifier()
```

In [76]:

```
dtc.fit(x,y)
```

Out[76]:

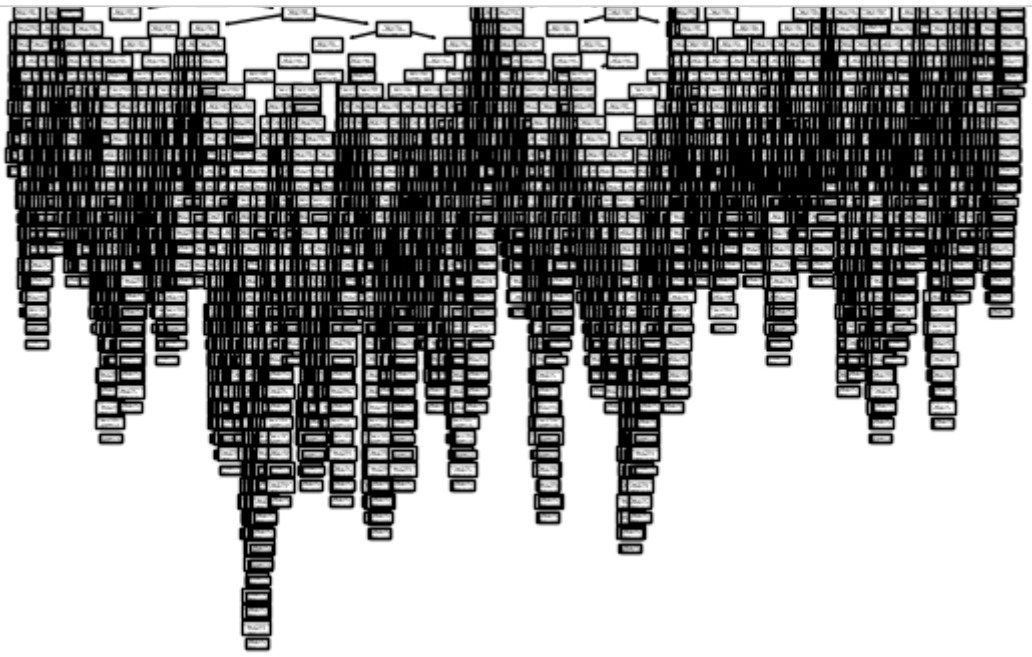
```
DecisionTreeClassifier()
```

In [77]:

```
from sklearn.tree import plot_tree
```

In [78]:

```
plot_tree(dtc)
```



In [79]:

```
dtc.score(x,y)
```

Out[79]:

```
0.9999692884125181
```

In [ ]:

```

```

In [80]:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

In [81]:

```
xtrain, xtest, ytrain, ytest = train_test_split(data1.drop("salary",axis=1),data1["salary
```

In [82]:

```
xtrain.shape, xtest.shape
```

Out[82]:

```
((24420, 14), (8141, 14))
```

In [83]:

```
dtc.fit(xtrain, ytrain)
```

Out[83]:

```
DecisionTreeClassifier()
```

In [84]:

```
plot_tree(dtc)
```

Out[84]:

```
[Text(0.6969518764040145, 0.9888888888888889, 'X[7] <= 0.5\ngini = 0.365
\nsamples = 24420\nvalue = [18544, 5876]'),
 Text(0.4524805814795371, 0.9666666666666667, 'X[4] <= 12.5\ngini = 0.494
\nsamples = 9904\nvalue = [5475, 4429]'),
 Text(0.28479793130767794, 0.9444444444444444, 'X[10] <= 5095.5\ngini =
0.441\nsamples = 6936\nvalue = [4658, 2278]'),
 Text(0.11525834385240051, 0.9222222222222223, 'X[4] <= 8.5\ngini = 0.416
\nsamples = 6603\nvalue = [4654, 1949]'),
 Text(0.0339908247659098, 0.9, 'X[11] <= 1760.0\ngini = 0.179\nsamples =
1115\nvalue = [1004, 111]'),
 Text(0.023113243370422434, 0.8777777777777778, 'X[0] <= 36.5\ngini = 0.1
65\nsamples = 1090\nvalue = [991, 99]'),
 Text(0.0025017273831931574, 0.8555555555555555, 'X[2] <= 29563.0\ngini =
0.068\nsamples = 311\nvalue = [300, 11]'),
 Text(0.0007624312024969622, 0.8333333333333334, 'X[3] <= 0.5\ngini = 0.5
\nsamples = 2\nvalue = [1, 1]'),
 Text(0.0003812156012484811, 0.8111111111111111, 'gini = 0.0\nsamples = 1
\nvalue = [0, 1]'),
```

In [85]:

```
dtc.score(xtrain,ytrain)
```

Out[85]:

```
0.9999590499590499
```

In [86]:

```
y_pred = dtc.predict(xtest)
```

In [87]:

```
print(classification_report(ytest, y_pred))
```

```
              precision    recall  f1-score   support

       <=50K       0.88      0.87      0.88      6176
        >50K       0.61      0.62      0.61      1965

    accuracy                           0.81      8141
   macro avg       0.74      0.75      0.74      8141
weighted avg       0.81      0.81      0.81      8141
```

In [88]:

```
dtc.score(xtest, ytest)
```

Out[88]:

```
0.8112025549686771
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: